



Product Build Report

How we researched, defined, and independently validated a production-ready church presentation & ministry-assistance platform — and the reasoning behind every major decision.

Status: Product definition complete · Independent audit PASSED · Awaiting go-ahead to build

PREPARED FOR

Project stakeholders & sponsors

PREPARED BY

SelahCue AI product & engineering team

DATE

23 July 2026

PHASE COVERED

Stages 1–4 (Discovery → Audit)

CLASSIFICATION

Internal

CONTENTS

What's in this report

About SelahCue — what it is and how it works (product overview)

1. **Executive summary** — where we are and why it matters
2. **How we work** — the disciplined method behind the work
3. **The product** — what SelahCue is and the gap it fills
4. **Stage 1** — Establishing a verified baseline
5. **Stage 2** — Discovery & research (the evidence)
6. **A key decision** — why we removed Text-to-Speech
7. **Stage 3** — The Product Requirements Document
8. **Stage 4** — Independent audit & how we passed it
9. **The MVP** — what ships first, and what waits
10. **Risks** — the honest picture and our mitigations
11. **What happens next** — the road to a shippable product
12. **Evidence index** — everything produced so far

WHY READ THIS

No code has been written yet — and that is deliberate. This report explains the substantial, evidence-based groundwork that de-risks the build before a single line of production code is committed. It is written for decision-makers, not engineers.

About SelahCue — what it is and how it works

SelahCue is a desktop application that runs the visual side of a church service — everything the congregation, the stage team and an online stream see on screen — with a companion phone/tablet app to control it, and optional AI assistance that helps but never takes over.

What it is

A cross-platform application for **Windows, macOS and Linux**, with a mobile controller for **Android and iPhone/iPad**. It is built for churches, conferences, worship events, livestream productions and sermon recording. One person — the media operator — drives the whole visual service from a single desktop, and the software keeps every screen showing exactly the right thing at the right moment.

The core idea: one live service, many screens

A service isn't shown on a single screen — it appears on several at once, and each needs something different at the same instant. SelahCue drives them all from one source of truth. At one moment in a single song, for example:

SCREEN	WHAT IT SHOWS AT THAT MOMENT
Main auditorium projector	Four lines of lyrics over a moving background
Stage / confidence monitor	The current line, the next line, a clock and the service timer — for the worship leader
Livestream output	Two lines of lyrics as a lower-third caption for the online audience
Lobby display	Announcements, continuing independently

How a service runs, step by step

1. **Before the service**, a coordinator builds a *service plan* — the ordered run-sheet of songs, scriptures, media and announcements — and SelahCue checks that every media file is present and ready.
2. **During the service**, the operator stages the next item in a *preview*, then sends it *live* with a click or keypress. What is being prepared is never accidentally shown to the congregation.
3. **Scripture** is found by reference or keyword and placed on screen in the chosen translation and layout.
4. **Timers** keep segments on schedule, with a configurable "TIME UP" signal that can be shown discreetly to the stage without ever appearing on the audience screen.
5. **Emergencies** — an instant blackout or a screen clear — are always one action away, and work with no network at all.

Control from a phone or tablet

Team members can control the service from a mobile device over the church's local Wi-Fi — advancing lyrics from the stage, running timers from the floor, or approving scripture from a tablet. Pairing is by QR code, connections are encrypted, and each person gets only the controls their role allows. Crucially, **the desktop is always in charge**: if a phone loses connection, the live service is completely unaffected.

Optional, responsible AI assistance

When enabled (in a later release), SelahCue can listen to the sermon and transcribe it, *suggest* the scripture the preacher is referencing for the operator to approve, and afterwards draft editable sermon notes. This assistance is always **operator-in-the-loop** — the software proposes, a human confirms — so an AI mistake can never reach the screen on its own.

The promise that underpins everything

SelahCue is built **offline-first** and **reliability-first**. The core of a service — slides, scripture, media, timers, blackout — keeps working with no internet, no cloud and no phone connected, and no failure of any single part (a lost screen, a failed AI service, a dropped network) can ever blank or clear what the congregation is seeing. It also saves continuously and restores its exact live state after a crash.

1 • Executive summary

SelahCue is a cross-platform church presentation and ministry-assistance application. Over four disciplined, independently-verified stages, we have taken it from an idea to a fully specified, audit-passed product definition that is ready to build.

4

Stages completed & approved

238

Traceable requirements
(177 functional + 27 non-functional + flows, risks, metrics)

0

Blocking defects at the independent audit

PASS

Final verdict of the fresh-context PRD audit

Where we are

We have finished the **product-definition half** of the project. Specifically, we have: inspected the environment and established a verified baseline; conducted broad, evidence-classified research into competitors, users, technology, licensing, privacy and the realistic limits of AI; written a complete Product Requirements Document (PRD) with stable IDs and testable acceptance criteria; and subjected that PRD to an **independent adversarial audit by fresh reviewers who did not write it**. The audit initially raised 17 significant issues; we resolved every one and the re-audit returned a clean **PASS**.

The most important decisions

- **A deliberately bounded first release.** The brief describes an enormous product. We scoped a realistic Minimum Viable Product — the reliable presentation core plus a secure mobile controller — and phased the ambitious AI features into later releases.
- **Honesty about AI.** Live transcription and "automatic scripture detection" are genuinely useful but imperfect. We designed them as an **operator-assisted** tool, never an autopilot, so a wrong AI guess can never embarrass a congregation.
- **Text-to-Speech removed.** On your instruction, and supported by our research, TTS was cut from the roadmap entirely (see Section 6).
- **Licensing safety first.** Only public-domain Bible translations are bundled; copyrighted translations and song lyrics remain the church's own licensed responsibility.

What's next

The next gate asks for your approval of the technical architecture and user-experience design (Stage 5), after which delivery is planned in the project tracker (Stage 6) and implementation begins in small, tested, independently-reviewed slices. **Nothing is deployed to production without your explicit, separate authorisation.**

SECTION 2

2 · How we work — and why it's built this way

SelahCue will run during live church services, conferences and livestreams — moments where software failure is highly visible and cannot be undone. That reality shapes our entire method. We optimise for **correctness and reliability**, not speed of typing.

Stage gates with human approval

Work advances through numbered stages. At the end of each, we stop, present evidence, and wait for your explicit decision to continue. Approval authorises *only* the next stage — never a blank cheque for the whole project.

WHY

It keeps you in control of scope, cost and direction at every step, and guarantees that expensive work (like writing code) never begins on an unapproved foundation.

Goal Contracts & measurable completion

Every stage and task runs against a written "Goal Contract" with Boolean, verifiable pass/fail criteria. Work is only "done" when those criteria pass *with evidence* — never because someone believes it looks finished. A small validation program checks each contract's structure automatically.

WHY

"It looks correct" is how defects reach production. Objective, checkable criteria remove wishful thinking from the definition of done.

Evidence, classified for honesty

Every research finding is tagged as **Observed** (we saw or measured it), **Documented** (a credible source states it), **Inferred** (reasoned, not proven), or **Unknown** (needs investigation). We deliberately record what we *don't* yet know.

WHY

It prevents marketing claims and guesses from masquerading as facts. When we say a number is unproven, you know exactly how much to trust it.

Independent verification

High-stakes claims — the requirements audit, security, quality — are checked by **fresh reviewers who did not do the original work**. No one signs off on their own homework.

WHY

Authors are blind to their own gaps. An independent adversarial reviewer catches what the writer cannot see — as proven in Stage 4, where 17 real issues were found and fixed.

One source of truth for delivery

Delivery work lives in your ClickUp workspace (the `BUILD CONTROL – Se1ahCue` task and its dedicated list). Durable product knowledge — this report, the PRD, research, decisions — lives in version-controlled documents. Progress is always recoverable without relying on chat history.

3 · The product & the gap it fills

SelahCue helps churches run the visual side of a service: presentation slides, song lyrics, Bible scripture, multiple screens, stage/confidence displays, lower-third graphics, and timers — with optional, responsible AI assistance layered on top, and a mobile phone/tablet that can control the desktop over the local network.

Why a new product is justified

Our competitor research (six products studied plus the wider production-tools ecosystem) found a real, unmet gap:

TODAY'S OPTIONS	THE TRADE-OFF CHURCHES ARE FORCED TO ACCEPT
ProPresenter	The most powerful, but Mac-first, ~US\$399/seat, a steep learning curve, and no Linux.
EasyWorship	Volunteer-friendly, but subscription-only and no Linux.
OpenLP / FreeShow / Quelea	Free and cross-platform, but utilitarian and light on professional AV routing and team roles.
PewBeam	Exciting AI-native newcomer, but appears focused on verse projection rather than a full production suite.

SelahCue's opportunity

Be genuinely cross-platform (Windows, macOS **and** Linux) with real mobile control, a volunteer-friendly experience with professional depth available underneath, native team roles and permissions, first-class integration with the broadcast ecosystem, and **responsible operator-controlled AI inside a complete production suite** — with offline-first reliability as an explicit promise.

SECTION 4

4 · Stage 1 — Establishing a verified baseline

What we did. Inspected the repository and tools, confirmed access to the ClickUp workspace, created the central `BUILD CONTROL – SelahCue` tracking task, wrote the top-level Goal Contract, documented a verified baseline, produced the research plan, and created the initial risk register.

What we found. A "greenfield" start — no existing code to reconcile, with the product brief as the single source of intent. We also discovered that two validation tools the process assumed did not yet exist, and flagged them openly as risks rather than quietly ignoring them (both have since been built).

WHY THIS STAGE EXISTS

Building on unverified assumptions is the most common cause of wasted effort. Establishing exactly what is (and isn't) true first means every later decision rests on ground truth, not guesswork.

Product naming. With your confirmation, the product is named **SelahCue** — "Selah" (a biblical term for a reflective pause) combined with "Cue" (a presentation cue).

5 · Stage 2 — Discovery & research

What we did. Six specialist researchers worked in parallel — competitor analysis, AI/transcription capabilities, licensing & compliance, cross-platform feasibility, security & privacy, and user roles & workflows — producing evidence-classified findings. A panel of four independent reviewers then audited the whole package (verdict: sound, with conditions we resolved). Where the brief demanded it, we did a follow-up study of dedicated transcription products.

The findings that shaped the product

Being honest about AI

This is where church-AI products most often over-promise. Our research is deliberately candid:

- **Live transcription** works well for clear, close-microphone speech but is materially less accurate in a real room (echo, PA bleed, accents, biblical names). Critically, the underlying technology can **invent plausible-sounding text during silence or music** — a serious risk in worship.
- **Automatic scripture detection** is **reliable when the preacher plainly states a reference** ("John 3:16"), **fallible for exact quotations**, and **unreliable for paraphrase**. We refuse to claim a single accuracy figure we cannot prove.

Design consequence

AI is an **operator assistant, not an autopilot**. By default, a human confirms every AI suggestion before it appears on screen, the deterministic parser is trusted over guesswork, and the system is tuned to avoid false positives. Every AI error is recoverable before the congregation ever sees it.

Licensing is a hard boundary

We can freely bundle only **public-domain** Bible translations (e.g. the World English Bible, ASV, Berean). Copyrighted translations (NIV, ESV, etc.) and copyrighted song lyrics **may not be bundled** — they remain the church's own licensed responsibility, which SelahCue supports rather than supplies. We also mapped video-codec patent and data-protection (NDPA/GDPR) obligations.

A technology reality check

The brief suggested a specific technology stack. Our feasibility work confirmed most of it is sound but surfaced one important correction — the lightweight app framework cannot double as the high-performance screen "composer," which needs a dedicated graphics engine. We captured ten precise technical questions ("spikes") to answer before committing the architecture in Stage 5, rather than guessing now.

Reliability as a promise, not a feature

The non-negotiable rule. The desktop application is always in charge. Core presentation — slides, local scripture, media, blackout, timers — must keep working with **zero** network, internet, mobile or AI availability, and **no failure of any component may blank or clear what's on screen as a side effect**. A dropped phone connection, a failed AI provider, or an unplugged screen must never take down the service.

An early security threat-model mapped the biggest risk — the local-network link between phone and desktop on shared church Wi-Fi — and the protections needed (encrypted, paired, permission-checked,

replay-protected connections). We also modelled 11 user roles and a 7-role mobile permission matrix.

SECTION 6

6 · A key decision — removing Text-to-Speech

The original brief included a Text-to-Speech (TTS) feature — the app reading text aloud. Partway through, you decided to hold off on it: *"I don't see it as a major requirement in the long run."*

Decision DEC-001 — TTS removed from the roadmap

TTS is reclassified from a "later release" feature to an explicit **non-goal**, revisited only if you request it in future. This is fully recorded and traceable.

Why this was the right call

Our own research had already rated TTS the **lowest-value** of the four AI features (a congregation reads the screen; it rarely needs the app to speak) and the **highest-risk** in a live room — audio feedback loops, microphone bleed, and the danger of speech accidentally playing through the main sound system. Removing it **reduces cost and risk** and simplifies the audio design, with no downside to the core product. The accessibility needs it might have served are met through other means (keyboard operation, high-contrast large text, screen-reader support).

Nothing was lost: the TTS research remains on file, so the decision is fully reversible if priorities change.

7 · Stage 3 — The Product Requirements Document

What we did. Turned the research into a complete, formal PRD: every capability written as a numbered requirement with a stable identifier, a release priority, and a **measurable, testable pass condition**. We built a validation program (the "PM artifact validator") that automatically checks the document is complete and internally consistent, and ran an independent review before the formal audit.

177	27	10	14
Functional requirements (FR)	Non-functional requirements (NFR)	Critical user journeys (FLOW)	Tracked risks (RISK)

WHY EVERY REQUIREMENT IS MEASURABLE

A vague requirement ("the app should be fast") cannot be tested, so it cannot be trusted. Each SelahCue requirement states a number or an observable behaviour — for example, *"a slide appears on the main screen within 150 milliseconds of the operator's command."* That is something quality engineers can objectively verify later.

What the PRD covers

- Service planning · slide and lyric editing · the full Bible scripture system · multiple independent screen outputs · stage and confidence displays · lower-third graphics · timers and a configurable "TIME UP" state · media playback · autosave and crash recovery · emergency clear and blackout · mobile
- discovery, pairing and secure control · live transcription · automatic scripture detection · AI sermon notes · production integrations · plus complete sections on permissions, data lifecycle, offline behaviour, security, privacy, accessibility, reliability, performance and licensing.

8 · Stage 4 — The independent audit

What we did. A panel of five fresh reviewers — none of whom wrote the PRD — audited it against a strict checklist: missing workflows, untestable criteria, unrealistic AI expectations, privacy/licensing/security/accessibility gaps, and over-sized scope. A lead reviewer then adjudicated a single verdict.

AUDIT ROUND	VERDICT	BLOCKING ISSUES	MAJOR ISSUES
First audit	Pass with conditions	0	17
After remediation (re-audit)	PASS	0	0 — all 17 discharged

WHY THIS IS THE MOST VALUABLE STAGE IN THE REPORT

The audit did its job: it found **17 genuine issues the authors had missed** — and it found them on paper, where a fix costs minutes, instead of in shipped code, where it could cost weeks or cause a live-service failure. This is exactly the return the disciplined method is designed to produce.

Examples of what the audit caught — and how we fixed it

THE GAP THE AUDITORS FOUND	HOW WE RESOLVED IT
What happens if the disk fills up during a long service? Autosave could silently fail.	Added a requirement for early low-disk warnings and protected space reserved for crash recovery.
If a crash is caused by the very content being restored, the app could crash-loop endlessly.	Added a "crash-loop breaker": after repeated crashes the app offers to start clean and disables the suspected item.
Removing TTS was clean in the main document but stale references lingered in supporting files.	Completed the removal everywhere, so no one builds a permission or workflow for a cancelled feature.
Even without TTS, the app's own media/alert audio could be re-recorded by the transcription microphone and trigger false detections.	Added a guard that pauses transcription while the app plays audio into the room.
Encrypted connections were specified for control commands but not for the live transcript sent to phones.	Broadened encryption to all data sent to mobile devices, including previews and transcripts.
Flashing "TIME UP" graphics posed a photosensitive-seizure risk on public screens.	Added a seizure-safety limit (no more than three flashes per second) and a reduced-motion option.

All 17 fixes were re-verified by fresh reviewers, confirming a clean PASS with no new problems introduced. The PRD is now cleared for the architecture and design stage.

9 · The first release (MVP) — what ships first, and why

The complete vision is large. Trying to build all of it at once is the single biggest risk to the project, so we defined a focused, genuinely useful **first release** and phased the rest.

In the first release

MVP The reliable presentation core on Windows, macOS and Linux, plus a secure mobile controller: service plans; slide, lyric and scripture presentation (public-domain Bibles); a main audience screen and a stage/confidence display; lower-third graphics; timers with "TIME UP"; media playback; autosave, crash recovery, emergency clear and blackout; mobile pairing with encrypted, permission-checked control of slides and timers; plus the security, privacy, accessibility and reliability foundations.

Deliberately held for later releases

RELEASE	ADDS	WHY LATER
R2	Extra screens, transparent lower-thirds, broadcast (NDI) output, diagnostics	Valuable but not essential to a working service; depends on performance proven by the Stage-5 spikes.
R3	Live transcription & transcript export	Needs the reliable core in place first; carries the most AI/privacy complexity.
R4	Automatic scripture detection	Builds directly on transcription (R3).
R5	AI sermon notes	Builds on the transcript; post-service, not live-critical.
R6	Production integrations, macros, right-to-left languages	Advanced/ecosystem features for power users.

Explicitly excluded

Non-goal Text-to-Speech (removed, DEC-001) · voice cloning · bundling any copyrighted Bible translation or song lyrics · copying any competitor's code or interface.

WHY BOUND THE MVP SO TIGHTLY

A smaller, excellent, reliable first release that churches can trust on a Sunday morning is worth far more than a sprawling, fragile one. Each later release adds value on a proven foundation.

10 • Risks — the honest picture

We track risks openly rather than discovering them late. The most significant:

RISK	WHY IT MATTERS	HOW WE'RE MANAGING IT
Scope breadth	The full product is effectively two products (a graphics suite and a secure control system) across five platforms.	Bounded MVP; phased releases; the desktop core ships before the mobile control plane.
Live reliability	A failure during a service is highly visible.	Offline-first core; strict rule that no component failure can blank the screen; fault-injection testing planned.
AI accuracy	Over-promising erodes trust.	Operator-confirmation by default; honest, documented limits; tuned to avoid false positives.
Performance on modest hardware	Some churches run older PCs.	Ten targeted technical spikes in Stage 5 to prove the graphics/performance approach before committing.
Licensing & privacy	Copyright and data-protection exposure.	Public-domain-only bundle; opt-in cloud with clear disclosure; sensitive legal items flagged for confirmation before the features that need them.

11 · What happens next

- **Stages 1–4 — Complete & passed**

Baseline, research, PRD, and independent audit (final verdict: PASS).

- **Stage 5 — Architecture & UX design** (next, needs your go-ahead)

Choose the technology with documented trade-offs, resolve the ten spikes, design the user experience and its failure/recovery states, and have architecture and design independently reviewed.

- **Stage 6 — Delivery planning**

Break the approved work into tracked, owned, dependency-ordered tickets in ClickUp; confirm the plan is ready before any code.

- **Stages 7–12 — Implementation**

Build in small, tested, independently-reviewed slices: the presentation foundation, then core presentation, mobile control, transcription, scripture intelligence, sermon notes, and hardening.

- **Stage 13 — Final verification**

Fresh-context code, quality, security and release-readiness checks before a "ready for release" recommendation.

Your control is preserved throughout. Each stage stops for your approval, and **production deployment requires its own separate, explicit instruction** — it never happens automatically.

12 · Evidence index — everything produced so far

All artefacts are version-controlled and traceable. This is the durable record behind the summaries above.

AREA	ARTEFACTS
Product definition	The complete PRD; the PM validation tool; the Stage-3 review and Stage-4 audit & re-audit reports.
Research evidence	Competitor matrix; AI/transcription capability assessment; provider trade-offs; licensing register; cross-platform feasibility study; adjacent-product study; consolidated evidence register; open-decisions log.
Users & security	Eleven personas with a role-permission matrix; normal, alternate and failure-recovery workflows; an early security threat model.
Governance	Goal Contracts for the build and each stage; the decision log (including the TTS decision); the living risk register; the ClickUp BUILD CONTROL — SelahCue task recording every gate.

SelahCue Product Build Report · Stages 1–4 · 23 July 2026 · Prepared by the SelahCue AI product & engineering team · Internal. This report summarises work completed to date; no production software has been deployed. Figures reflect the product definition as audited; performance targets are provisional pending the Stage-5 technical spikes.